

Robot Framework With Python Advanced Capstone Project (UI + API + Integration)

Goal:

The objective of this capstone project is to evaluate your knowledge on your respective tracks on the aspects of end-to-end Software Development Life Cycle (SDLC) principles with a strong emphasis on modern, AI-assisted Quality Engineering practices. The evaluation assesses candidates' skills across requirements analysis, manual test design, UI and API testing, hybrid integration validation, automation engineering, and advanced AI-driven enhancements, using a real-world banking application.

This project is designed to measure how effectively participants can:

- Analyze functional and integration requirements
- Design structured test plans, scenarios, test cases, and traceability (RTM)
- Validate data consistency across UI and API layers
- Build scalable Robot Framework with Python automation suite (UI, API, and hybrid)
- Apply AI-assisted techniques, agentic workflows, and MCP-based capabilities
- Integrate automation with CI/CD and reporting pipelines
- Demonstrate engineering discipline, clarity of thought, and industry readiness

Note:

We will also review the logic and uniqueness of the coding standards. The entire project will be rejected if any malpractice is identified or if the content is generated entirely using AI.

Application: Parabank

Description: ParaBank is a demo site created solely to simulate a realistic online banking experience and is not a real bank

UI: <https://parabank.parasoft.com>

API Swagger: <https://parabank.parasoft.com/parabank/api-docs/index.html>

Validation: UI and API validations of account creation and fund transfer

Technical implementation: Robot Framework + Python

Framework Model: POM + API Fixtures + Parallel Execution

Section 1 — Requirements, Manual Test Plan, Test Scenarios, Test Cases, RTM

Objectives:

As a Quality Engineer, please focus on the following

- Analyzing the functional and integration requirements,
- Creating a detailed manual test plan,
- Defining high-value test scenarios
- Designing comprehensive test cases,
- Establishing requirement traceability to ensure complete coverage.

1.1 Functional Requirements

Req ID	Requirement
FR-01	Create new account via UI
FR-02	GET accounts list via API
FR-03	Validate new account exists in API
FR-04	Validate account type and details in API
FR-05	Transfer funds via UI
FR-06	Validate updated balances using API
FR-07	Validate transfer applied correctly (before/after values)
FR-08	Validate UI messages (success, confirmation)
FR-09	Validate negative scenarios

1.2 Manual Test Plan :

Purpose

Validate end-to-end account lifecycle and fund transfer flows across UI and API, ensuring data consistency, correctness, and error handling. Scenarios should cover both Functional (UI, API) and Hybrid (UI + API)

Scope

- Account creation via UI
- Account retrieval & validation via API
- Fund transfer via UI
- Balance and transaction validation via API
- Positive and negative scenarios
- UI message validations
- Hybrid scenarios (UI + API)

Sample Test Scenarios only for reference – Account Creation via UI (FR-01)

TS-01: Successful Account Creation

Verify that a new account can be created successfully using valid inputs.

TS-02: Mandatory Field Validation

Verify validations for required fields during account creation.

TS-03: Field Level Validation

Verify format, length, and boundary validations for account input fields.

Sample Test Cases

UI Based

Field	Details
Scenario ID	TS-01
Test Case ID	TC-AC-UI-01
Test Case Name	Create New Account with Valid Data
Description	Verify that a user can create a new account using valid details
Pre-conditions	User is logged in Account creation page is available

Test Steps	1. Go to Create New Account page 2. Enter valid Account Name 3. Select Account Type (Savings / Checking) 4. Enter valid Initial Deposit 5. Click Submit / Create Account
Expected Result	Account is created successfully Success message is shown Account number / ID is displayed

API Based

TC-API-01: Validate New Account Exists via API

Field	Details
Scenario ID	TS-03
Test Case ID	TC-API-01
Test Case Name	Validate New Account Exists via API
Description	Verify that the account created via UI is available in the API
Pre-conditions	Account is created successfully via UI Valid API access is available
Test Steps	1. Capture Account ID created from UI 2. Call GET Accounts API 3. Search for the captured Account ID in API response
Expected Result	API response is successful (200 OK) New Account ID is present in the response

UI + API (Hybrid) Based

TC-E2E-01: Create Account via UI and Validate via API

Field	Details
Scenario ID	TS-01
Test Case ID	TC-E2E-01
Test Case Name	Create Account via UI and Validate via API
Description	Verify account creation using UI and validate the same using API
Pre-conditions	User is logged in Account creation page is accessible API endpoints are available
Test Steps	1. Create a new account using UI 2. Capture generated Account ID from UI 3. Call GET Accounts API 4. Validate account details using Account ID
Expected Result	Account is created successfully via UI Account exists in API response Account details match UI data

1.3 Requirement Traceability Matrix (RTM)

The Quality Analyst should be capable of ensuring that every generated test case is accurately mapped to its corresponding test scenario, and each test scenario is linked to one or more dependent requirements—thereby guaranteeing complete requirement coverage and achieving end-to-end traceability.

Sample RITM:

FR	Scenario(s)	Test Case(s)
FR-01	TS-01	TC-01
FR-02	TS-01, TS-03	TC-02

FR-03	TS-03, TS-04	TC-02
FR-04	TS-03, TS-04	TC-02
FR-05	TS-06	TC-04
FR-06	TS-07	TC-05
FR-07	TS-07, TS-08	TC-05
FR-08	TS-02	TC-01
FR-09	TS-09	TC-NEG

Test Execution

- Test cases should be executed based on the finalized test plan and defined scope.
- Execution should cover – UI, API and Hybrid test cases
- Each test case should be marked as:
 - Pass
 - Fail
 - Blocked (if there are environment or dependency issues)
- Evidence such as screenshots, logs, and API responses should be attached for failed test cases.



Test_Execution_and
_Reporting_Templat

Test Reporting

- Test reports should include:
 - Total number of test cases executed
 - Passed, Failed, and Blocked test cases
 - Defect summary
- A final Test Summary Report should be prepared at the completion of testing.

Defect Raising and Tracking

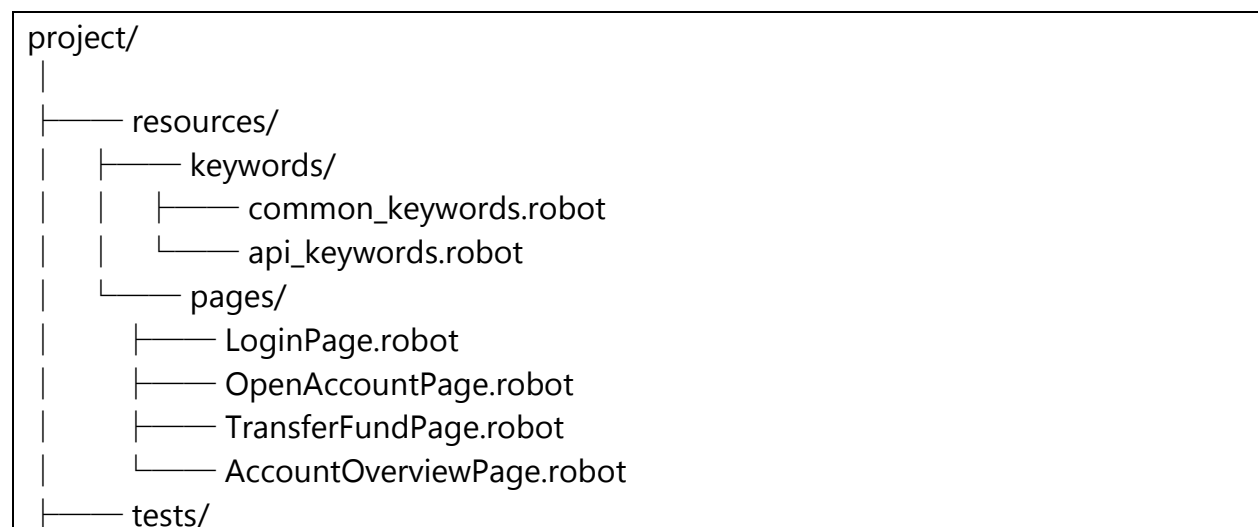
- Any mismatch between expected and actual results should be logged as a defect.
- Each defect should include:
 - Defect ID
 - Related Test Case ID
 - Clear defect description
 - Steps to reproduce
 - Expected and Actual results
 - Screenshots or logs as evidence
- Defects should be classified based on:
 - Severity (Critical / High / Medium / Low)
 - Priority (P1 / P2 / P3)

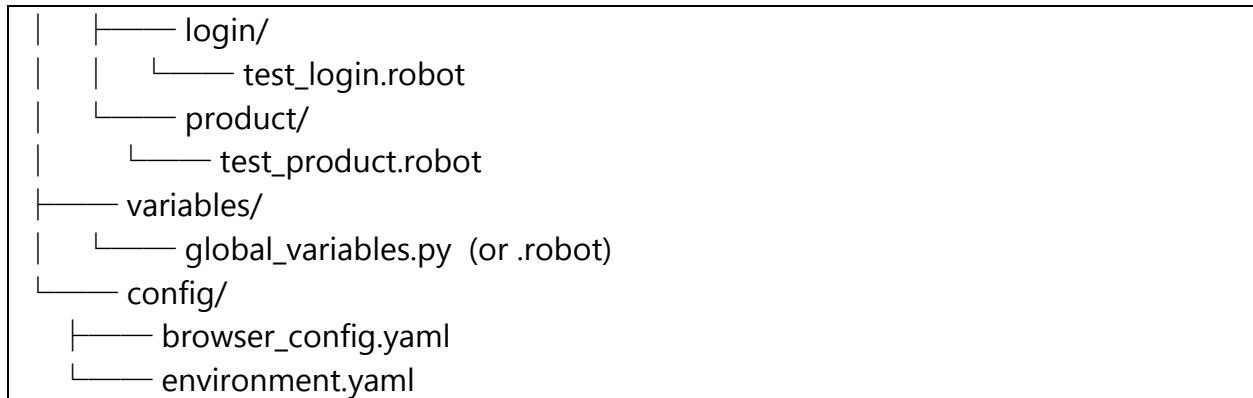
Section 2 — Robot Framework Automation (UI + API + Hybrid)

Objectives:

As a Quality/QA Test Automation Engineer, please build a scalable Robot Framework with Python automation , implement UI and API test scripts using best practices, create hybrid end-to-end flows, and integrate structured logging and reporting to ensure a robust, maintainable, and extensible test suite.

2.1 Project Structure (Robot Framework Best Practices)





2.2 UI Automation (POM) Framework includes:

Robot Framework-specific best practices:

- ✓ Prefer Smart Waiting Instead of Sleep
- ✓ Use Stored Authentication State
- ✓ Create a Fresh Browser Context for Every Test
- ✓ Build Reusable Python-Based Page Objects

Example Page Objects:

- LoginPage
- OpenAccountPage
- TransferFundsPage
- AccountOverviewPage

2.3 API Automation (APIRequestContext)

API Implementation:

- Define API fixture or reusables with authentication
- Create specs for API manual test cases
- All requests and responses should be logged

2.4.2.7 Logging + Reporting

- ✓ Robot Framework's Built-In Logging
- ✓ Using Python's Built-In Logging Module for API
- ✓ Allure Report Integration with screenshots
- ✓ Custom HTML Logs (Optional Advanced Use)

Section 3 — Advanced Topics, Agentic Layer, MCP, AI-Assisted QE, Performance, CI/CD & Final Deliverables

Objectives:

As a Quality/QA Test Automation Engineer, please enable advanced capabilities such as parallel execution, CI/CD integration, AI-assisted quality enhancements, agentic workflows, MCP-based intelligent locator handling, performance checks, and finalize all deliverables to complete an enterprise-grade automation solution.

3.1 Advanced Robot Framework Execution & Test Organization

3.1.1 Advanced Parallel Execution

- Use Pabot for High-Performance Parallel Execution
- Leverage Test Sharding Using Pabot Options
- Use Separate Browser Contexts/Sessions Per Process
- Add Suite/Test-Specific Retry Logic Using Listeners or ReRunFailed

3.1.2 Test Organization Enhancements

- **Split tests into:**
 - /tests/e2e
 - /tests/performance-lite
- **Implement:**
 - Tagging (@smoke, @regression, @api, @e2e)
 - Data-driven tests using .csv/JSON

3.2 CI/CD Integration (Jenkins/Github)

1. Collect artifacts:
 - Screenshots
 - Videos
 - Traces
 - Allure report
2. Publish Summary report including HTML report

3.3 AI-Driven Quality Engineering Extensions

1. AI-Generated Test Data

- Use Coding assistant like Github Copilot and create utility to generate random data and use it in automation scripts
- 2. AI Failure Analysis
 - Use Coding assistant like Github Copilot generate Analysis html report with following details,
 - Failure count
 - Failure reason
 - Historical analysis
- 3. Enhance coverage using AI and then automate them.

3.4 MCP – Model Context Protocol Integration

Implement script generation and execution using RobotMCP

3.5 Performance Engineering (Basic)

Lightweight Checks:

- Capture UI page load times using performance.timing
- Capture API response times
- Fail if API > 2 seconds (per requirement)

Add mini performance suite:

- Basic throughput validation
- Repeated GET accounts 20 times asynchronously

3.7 Final Capstone Deliverables

3.7.1 Functional QA Deliverables

- ☐ Test Plan
- ☐ Test Scenarios
- ☐ Test Cases
- ☐ Manual Test Execution Reports
- ☐ Defect Report
- ☐ RTM

3.7.2 Automation QA Deliverables

Code Assets

- ☐ Robot Framework E2E Integrated Test Suite (UI → API + API → UI)

- ☐ AI-driven enhancements
- ☐ Agentic + MCP addons

Reports

- ☐ Allure report
- ☐ Robot Framework In Built Reports
- ☐ Screenshots, videos, traces

DevOps Deliverables

- ☐ CI/CD pipeline YAML
- ☐ Parallel execution config
- ☐ Environment configs